

Detection Lab Operator's Manual

A practical guide to running the CyberSecurity_Lab detection simulations with Sysmon, EDR, Procmon, and packet capture (Wireshark / tcpdump) — properly, efficiently, and ethically.

0. Read this first — what these labs are

This repository contains four **safe, self-contained blue-team training simulations**. Each one reproduces the *observable traces* an attack technique leaves behind so a defender can practise **detecting** it. They are deliberately **not** attack tools:

- All traffic stays on the **loopback interface** (`127.0.0.1`) by default. Reaching another machine requires an explicit `--allow-lan` flag that permits **private/lab ranges only** (RFC1918, CGNAT `100.64/10`, link-local, IPv6 ULA, loopback). **Any public/routable address is refused**.
- There is **no source-IP spoofing** anywhere; simulated "attacker" IPs are just text tags drawn from the RFC 5737 documentation ranges (e.g. `203.0.113.0/24`).
- The "malware" writes only **clearly-labelled dummy files**; the "keylogger" never reads a keyboard; injection payloads are **logged, never executed**; worker counts, durations, and response sizes are **hard-capped**.
- Every step is appended to an `events.jsonl` **answer key** in the lab directory, and each lab has a built-in `report` (timeline + IOCs) and `detect` (hunting guidance) command.

The goal is always the same: **run a scenario with your detection tooling armed, then compare what your tools caught against the lab's own answer key**.

1. Rules of engagement (ethics & legality)

These simulations are harmless, but the **tooling and mindset** around them are the same you would use against real systems. Treat every session as if it were a real engagement.

- 1 **Authorization.** Run these only on hardware/VMs and networks **you own or are explicitly, in-writing authorized to test**. Unauthorized access to or interference with computers is a crime in most jurisdictions (e.g. the US Computer Fraud and Abuse Act, the UK Computer Misuse Act, and equivalents). The safety rails here do not grant you permission — *you* are responsible for staying inside your own lab.
- 2 **Isolate the lab.** Prefer a dedicated VM or an isolated lab network. Keep the default loopback mode unless you specifically need two hosts, and never pass `--allow-lan` on a network you do not fully control. Do **not** run the DoS/flood lab against shared infrastructure, even on a private LAN — use an isolated victim VM so you never degrade something someone else relies on.
- 3 **No public targets, ever.** The labs refuse public addresses by design; do not attempt to defeat that. If a command errors with "refusing to use ... not a private/lab address," that is the safety rail working as intended.
- 4 **Handle the data responsibly.** `events.jsonl`, `.pcap` captures, and EDR exports are lab data, but build the habit now: store them where only you can read them, and delete them when the exercise is over (`reset` per lab).

- 5 **Be honest in write-ups.** If your tooling *missed* something, record that. The point of the exercise is to find and close detection gaps, not to produce a clean-looking report.

One-line test: *If you could not comfortably show this session to the owner of the network you are on, you should not be running it here.*

2. Prerequisites

Need	Detail
Python	3.9+ (standard library only — nothing to <code>pip install</code>)
OS for host telemetry	Windows 10/11 for Sysmon and Procmon
OS for <code>tcpdump</code>	Linux/macOS (or WSL)
Cross-platform	Wireshark works on Windows/Linux/macOS
Privileges	Admin/root only for installing Sysmon and for packet capture; the labs themselves need no elevation

Run a lab's tests to confirm your Python works before you start (from a lab folder): `py -m pytest -q`.

The four labs and their default listeners:

Lab	Folder	Protocol / port	Simulates	MITRE
Gift-Card IR	gift_card_lab/	TCP/HTTP 8765	Malicious lure → payload → C2 beacon → exfil	T1036, T1204, T1071, T1041
DDoS	ddos_lab/	TCP/HTTP 8768	Volumetric & slow-rate denial of service	T1498, T1499
Amplification	amplification_lab/	UDP 8769	UDP reflection/amplification factor	T1498.002
Web-Attack	web_attack_lab/	TCP/HTTP 8770 (+16 sensor ports)	Credential stuffing, enumeration, injection probes, port scan	T1110, T1595, T1190

3. The observability toolkit — one-time setup

You do not need every tool for every lab. Section 5 says which tool matters for which lab. Set up the ones you have.

3.1 Sysmon (Windows host telemetry) — best for the Gift-Card lab

Sysmon writes rich process/file/network events to the Windows event log.

Install (Admin PowerShell), using a community base config:

```
# Download Sysmon from Microsoft Sysinternals, then:
sysmon.exe -accepteula -i sysmonconfig.xml
```

A good starting config is SwiftOnSecurity's `sysmon-config` or Olaf Hartong's `sysmon-modular`. **Verify** it is running: `Get-Service Sysmon*`.

Where events land: Event Viewer → Applications and Services Logs → Microsoft → Windows → Sysmon → Operational.

Event IDs you will use here:

EID	Meaning	Why it matters in these labs
1	Process Create	The parent→child <code>python.exe</code> chain (Gift-Card)
3	Network Connect (TCP+UDP)	Loopback callbacks, beacon, scan connects
11	File Create	Dummy files written into the lab dir
15	FileCreateStreamHash	Mark-of-the-Web / alternate data streams (real-world analogue)
22	DNS Query	Not triggered here (labs use IPs), but hunt it in the wild

3.2 EDR / Microsoft Defender for Endpoint

If you have Defender for Endpoint (or a similar EDR), use **Advanced Hunting**. The three tables that map onto these labs:

- `DeviceProcessEvents` — process creation (the interpreter chain).
- `DeviceFileEvents` — file writes into the user profile / lab dir.
- `DeviceNetworkEvents` — outbound connections (beacon, exfil, scan).

Ready-to-paste KQL sits in each lab's `detect` output and in Section 5.

For any other EDR, the concept is identical: filter on the `python.exe` process tree, the files it writes, and the connections it opens, in the run's time window.

3.3 Procmon (Process Monitor) — fine-grained host activity

Best when you want to *see every operation* rather than curated events.

- 1 Launch Procmon (Admin). It starts capturing immediately.
- 2 **Filter aggressively** (Ctrl+L) or you will drown in noise. Useful rules:
 - Process Name is `python.exe` → Include
 - Operation is Process Create → Include
 - Operation is TCP Connect → Include
 - Path contains `gift_card_ir_lab` → Include
- 1 Run the lab, then stop the capture (Ctrl+E) so the view stops moving.
- 2 Optionally back the capture with a file (File → Backing Files) for long runs.

3.4 Wireshark (packet capture, GUI, cross-platform)

Capturing loopback traffic is the catch:

- **Windows:** install **Npcap** with the "*Support loopback traffic*" option, then capture on the "**Adapter for loopback traffic capture**" interface.

- **Linux:** capture on `lo`. **macOS:** capture on `lo0`.

Apply a **display filter** scoped to the lab's port so you see only lab traffic (filters per lab in Section 5). The **Statistics** → **I/O Graph** and **Statistics** → **Conversations** views are invaluable for the DoS/amplification labs.

3.5 tcpdump (packet capture, CLI, Linux/macOS)

Scope every capture to the loopback interface and the lab's port, and write a `.pcap` you can open later in Wireshark:

```
sudo tcpdump -i lo -n port 8765 -w giftcard.pcap
# open later:  wireshark giftcard.pcap
```

Per-lab filters are in Section 5. Stop with Ctrl+C; the summary line tells you how many packets were captured.

4. The efficient workflow (do this every time)

A repeatable loop keeps sessions fast and your findings clean:

- 1 **Pick one scenario.** Run a single attack type per pass; mixing them muddies the telemetry.
- 2 **Baseline (optional but recommended).** Start the victim/server and let your tools capture ~10 s of *idle* traffic so you know what "normal" looks like.
- 3 **Arm the tooling.** Start Sysmon (already running), Procmon capture, and your packet capture **before** you launch the attack. Scope packet filters to the lab port up front.
- 4 **Note the time and run id.** Each lab stamps every event with a `run_id` (visible in `report`). Jot the wall-clock start time — you will pivot your tools to that window.
- 5 **Run the scenario** (the one-click `run` mode is easiest; two-window mode lets you watch the server separately).
- 6 **Stop captures** so the views stop scrolling.
- 7 **Compare against the answer key.** Run the lab's `report` and `detect`, then check: *did each recorded event show up in my tooling?* Every gap is a detection you need to build.
- 8 **Reset.** `py <lab>.py reset` wipes the lab dir so the next pass is clean. Delete or archive your `.pcaps`.

Correlation tip: the `run_id` ties every lab event to one exercise. Use the event timestamps (UTC) to line up `report` against Sysmon/EDR/pcap timelines.

5. Per-lab playbooks

Each playbook lists the commands, what each tool should show, and an **event** → **telemetry** map so you can confirm your coverage.

5.1 Gift-Card IR lab — masquerading lure, C2 beacon, exfil

Folder: `gift_card_lab/` · **Port:** TCP 8765 · **MITRE:** T1036 / T1204 (lure & user execution), T1071 (C2), T1041 (exfil over C2).

This is the **host-telemetry** lab — Sysmon, EDR, and Procmon do the heavy lifting; packet capture is secondary (traffic is loopback HTTP).

Run it (one click):

```
cd gift_card_lab
py gift_card_lab.py run --announce --auto-serve      # full lure-payload chain
py gift_card_lab.py beacon --beacons 20 --interval 2 --jitter 0.5  # C2 check-ins
py gift_card_lab.py exfil --exfil-bytes 262144 --chunk 16384      # staged upload
py gift_card_lab.py report                                # timeline + IOC answer key
py gift_card_lab.py detect                                # built-in hunting guide
py gift_card_lab.py reset                                # clean up
```

For beacon/exfil, start the server first in another window (`py gift_card_lab.py serve`) so there is a listener to check in to.

What each tool should catch:

- **Sysmon EID 1 (Process Create):** the parent `python.exe` spawning a child `python.exe ... child ...`. The parent→child interpreter chain is the core lesson.
- **Sysmon EID 11 (File Create):** `gift_card.txt`, `dummy_input_events.json`, and `received_telemetry_*.json` appearing under `~/gift_card_ir_lab`.
- **Sysmon EID 3 (Network Connect):** connections to `127.0.0.1:8765`. During beacon, these repeat at a near-constant interval — **graph the timestamps** and the periodicity is the signature. During exfil, watch **outbound bytes climb** in fixed-size chunks.
- **Procmon:** filter to `python.exe + Process Create + TCP Connect + Path contains gift_card_ir_lab` to see the whole chain in order.
- **Wireshark / tcpdump (port 8765):** GET `/gift-card` (the "download"), POST `/lab-telemetry` (the payload phoning home), repeated GET `/c2-beacon` (beaconing), and POST `/exfil` (the staged upload).

Defender Advanced Hunting (KQL):

```
DeviceProcessEvents
| where InitiatingProcessFileName in~ ("python.exe","wscript.exe","powershell.exe")
| where FolderPath has_any ("\\Downloads\\","\\Temp\\","\\Desktop\\")
| project Timestamp, DeviceName, FileName, ProcessCommandLine,
    InitiatingProcessFileName, InitiatingProcessCommandLine
```

Beacon periodicity sketch:

```
DeviceNetworkEvents
| summarize hits=count(), span=max(Timestamp)-min(Timestamp)
    by RemoteIP, InitiatingProcessFileName
| where hits > 10    // then inspect the inter-arrival regularity
```

Event → telemetry map:

Lab event	You should see
gift_card_file_downloaded	File create (+ Mark-of-the-Web in the real world)

Lab event	You should see
child_process_launch_requested/ child_process_started	Sysmon EID 1 parent→child chain
dummy_event_file_created	Sysmon EID 11 file create in user profile
loopback_request_sent	Sysmon EID 3 network connect (127.0.0.1 here)
c2_beacon_sent (evenly spaced)	Periodic-beacon detection (T1071)
exfil_chunk_sent (growing)	Outbound data-volume anomaly (T1041)

5.2 DDoS lab — volumetric & slow-rate denial of service

Folder: `ddos_lab/` · **Port:** TCP 8768 · **MITRE:** T1498 / T1499.

This is a **network + service-metrics** lab. Packet capture and the victim's own rate/connection counters matter most; host process telemetry is less useful.

Run it:

```
cd ddos_lab
py ddos_sim.py run --attack volumetric --announce      # request-rate spike
py ddos_sim.py run --attack slowloris --announce      # connection exhaustion
# other styles: cachebust, connflood, bodyflood, rudy, slowread
py ddos_sim.py report
py ddos_sim.py detect
py ddos_sim.py reset
```

Two-window mode: `py ddos_sim.py serve` (victim, leave running) then `py ddos_sim.py flood --attack volumetric` in a second window. Tune with `--workers` (default 50, max 256) and `--duration`.

What to watch:

- **Wireshark / tcpdump** (port 8768): for **volumetric/cachebust**, a burst of many short-lived connections and a high request rate — use **Statistics** → **I/O Graph** to see the spike, and **Conversations** for the source fan-out. For **slowloris/rudy/slowread**, few connections that **open and stay open** without completing — filter `tcp.flags.syn==1` for opens and look for the absence of clean completions.
- **The lab's own metrics:** `report` shows requests/sec, peak concurrency, and unique sources; the server prints mock alerts when it crosses `--alert-rps` or `--alert-conns`. This stands in for a WAF/load-balancer alarm.

```
sudo tcpdump -i lo -n 'port 8768 and tcp[tcpflags] & tcp-syn != 0' -w ddos.pcap
```

Detection focus: rate anomalies (volumetric) vs. concurrency that climbs and holds with requests that never complete (slow-rate). Different signatures, same outcome — service latency collapse.

5.3 Amplification lab — UDP reflection factor

Folder: `amplification_lab/` · **Protocol:** UDP 8769 · **MITRE:** T1498.002.

A concept lab: a **small query provokes a large response**, and you measure the amplification factor. There is no spoofing, so you cannot aim it at anyone — the big response comes straight back to you.

Run it:

```
cd amplification_lab
py amp_lab.py run --factor 50 --queries 200 --announce
py amp_lab.py report          # amplification-factor summary
py amp_lab.py detect
py amp_lab.py reset
```

What to watch — this is a packet-capture lab:

- **Wireshark:** filter `udp.port == 8769`. Compare the **query packet length** to the **response packet length** — the ratio *is* the amplification factor. Statistics → Conversations → UDP shows bytes-out vs bytes-in per flow.
- **tcpdump:** `sudo tcpdump -i lo -n udp port 8769 -w amp.pcap` then inspect packet sizes (`-v` shows length).

```
udp.port == 8769 && udp.length > 200          # Wireshark: the oversized responses
```

Detection focus: small, uniform requests answered by disproportionately large responses to/from one host — in the wild, correlate with a victim receiving unsolicited large UDP responses it never queried.

5.4 Web-Attack lab — credential stuffing, enumeration, injection, port scan

Folder: `web_attack_lab/` • **Port:** TCP 8770 (+16 sensor ports above it) • **MITRE:** T1110 (brute force), T1595 (active scanning), T1190 (exploit public-facing app).

An **access-log + connection-telemetry** lab. The victim only *logs* what it receives — injection payloads are recorded, never executed.

Run it:

```
cd web_attack_lab
py web_attack_lab.py run --attack credstuff --announce # failed-login burst
py web_attack_lab.py run --attack enum --announce    # 404 dir-busting
py web_attack_lab.py run --attack inject --announce  # logged injection probes
py web_attack_lab.py run --attack portscan --announce # TCP port fan-out
py web_attack_lab.py report
py web_attack_lab.py detect
py web_attack_lab.py reset
```

Tune with `--workers` (default 20) and `--rounds`. Alert thresholds: `--alert-401` (failed logins/sec), `--alert-404` (404s/sec), `--alert-scan` (distinct ports one source touches).

What to watch:

- **credstuff:** Wireshark filter `http.request` on port 8770 — a burst of POST `/login` with 401 responses. Hunt many failed logins from one source in a short window.
- **enum:** many 404s across guessed paths. Filter `http.response.code == 404`.
- **inject:** GET/POST carrying SQLi/XSS-looking strings in the URI or body — present in the request, and the report's `injection_probes` counter tallies them. In the wild this is a WAF signature.
- **portscan:** **Sysmon EID 3** and Wireshark shine here — one source touching many destination ports (8770 and the 16 sensor ports above it, plus some closed ones). Filter `tcp.flags.syn == 1 && tcp.flags.ack == 0` to see the scan's SYNs.

```
sudo tcpdump -i lo -n 'port 8770 or portrange 8771-8790' -w webscan.pcap
```

Detection focus: rate + fan-out. Brute force = many auth failures from one source; enumeration = many 404s; scan = one source, many ports.

6. Reading the answer key

Every lab shares the same three read-out commands:

- `report` — the timeline of recorded events plus an IOC/metrics summary. This is your ground truth. Work through it line by line and confirm each event has a matching artifact in your tooling.
- `detect` — technique-specific hunting guidance (indicators, Sysmon Event IDs, and copy-paste KQL) for what a defender should look for.
- `reset` — deletes the lab working directory. It refuses to delete anything that does not look like a lab dir (your home, a repo, a filesystem root), so a mistyped `--lab-dir` cannot nuke real data.

The exercise is a loop: **run** → **capture** → `report/detect` → **find the gaps** → **build the detection** → `reset` → **repeat**.

7. Cross-host mode (two VMs) — only on a lab network you own

Three of the labs (`gift_card`, `amplification`, `web_attack`) accept `--allow-lan` to run the two halves on separate machines. This permits **private/lab ranges only**; public addresses are always refused.

- **Server side:** `serve --allow-lan --bind 0.0.0.0` (Gift-Card) or the lab's equivalent, then find the box's IP.
- **Client side:** `point --server/--target <lab-ip>` and add `--allow-lan`.

If your VMs are on different hypervisors, they must be **bridged onto the same physical LAN** to reach each other (host-only/NAT will not span two hosts). Confirm with `ping` first, and **only do this on a network you fully control**. The DoS lab is intended for a single isolated victim VM — do not flood a shared LAN.

8. Troubleshooting

Symptom	Fix
No loopback packets in Wireshark (Windows)	Reinstall Npcap with <i>"Support loopback traffic"</i> ; capture on the loopback adapter
<code>port already in use</code>	Another lab/instance is bound; <code>reset</code> , close it, or pass a different <code>--port</code>
<code>py</code> not found	Use <code>python</code> instead, or install the Python launcher
"refusing to use ... not a private/lab address"	Working as intended — the target is public; use loopback or a private lab IP
Antivirus/Defender flags <code>python.exe</code> behavior	Expected — you are generating attack-like telemetry. Run in an isolated VM and, if needed, add a scoped exclusion for the lab dir only

Symptom	Fix
Beacon/exfil "could not reach the lab server"	Start <code>serve</code> first (in another window or on the server VM)
Cross-host halves can't reach each other	Bridge both VMs to the same LAN; <code>ping</code> first; pass <code>--allow-lan</code> on both sides

9. Cleanup checklist

- [] `py <lab>.py reset` for each lab you ran.
- [] Stop Procmon and packet captures.
- [] Archive or delete `.pcap` files and any EDR/Sysmon exports.
- [] If you used `--allow-lan`, confirm the lab servers are stopped.

Appendix A — Quick command reference

Lab	Serve	Attack / run	Read-out
Gift-Card	<code>serve</code>	<code>run --auto-serve · beacon · exfil</code>	<code>report · detect · reset</code>
DDoS	<code>serve</code>	<code>flood --attack <type> · run --attack <type></code>	<code>report · detect · reset</code>
Amplification	<code>serve</code>	<code>reflect · run --factor N --queries N</code>	<code>report · detect · reset</code>
Web-Attack	<code>serve</code>	<code>attack --attack <type> · run --attack <type></code>	<code>report · detect · reset</code>

Appendix B — Packet-capture filter cheat-sheet

Lab	Wireshark display filter	tcpdump
Gift-Card	<code>tcp.port == 8765</code>	<code>-i lo port 8765</code>
DDoS	<code>tcp.port == 8768</code>	<code>-i lo port 8768</code>
Amplification	<code>udp.port == 8769</code>	<code>-i lo udp port 8769</code>
Web-Attack	<code>tcp.port == 8770 or (tcp.port >= 8771 and tcp.port <= 8790)</code>	<code>-i lo 'port 8770 or portrange 8771-8790'</code>

(On Windows use the loopback adapter; on Linux `lo`, on macOS `lo0`.)

Appendix C — Sysmon Event IDs used here

EID	Event	Seen in
1	Process Create	Gift-Card (parent→child chain)
3	Network Connect (TCP/UDP)	All labs (callbacks, beacon, scan)
11	File Create	Gift-Card (dummy artifacts)
15	FileCreateStreamHash	Real-world Mark-of-the-Web analogue
22	DNS Query	Not fired here (labs use IPs) — hunt in the wild

Appendix D — MITRE ATT&CK mapping

Lab	Techniques
Gift-Card	T1036 Masquerading · T1204 User Execution · T1071 App-Layer Protocol (C2) · T1041 Exfil Over C2 Channel
DDoS	T1498 Network DoS · T1499 Endpoint DoS (.001 slow-rate, .002 volumetric)
Amplification	T1498.002 Reflection Amplification
Web-Attack	T1110 Brute Force · T1595 Active Scanning · T1190 Exploit Public-Facing Application

These labs are for authorized detection training only. You are responsible for using them lawfully and on systems you own or are permitted to test.